



PCS 3838 - Inteligência Artificial

Informações Gerais

1. As entradas são sempre separadas por espaço. Por exemplo, 4 6 indica que o programa deve ler dois valores (4 e 6).
2. Sempre que possível, evite modificar a leitura já disponibilizada nos códigos fornecidos.
3. Atente-se aos exemplos fornecidos nas tabelas. Esses exemplos mostram o padrão de leitura e exibição que cada programa deve conter.
4. Você pode utilizar pacotes python disponíveis no ambiente VPL como, por exemplo, numpy, scipy, etc.
5. A menos que indicado o contrário, devido a questões numéricas do VPL, exiba apenas a parte inteira dos valores. Por exemplo, 3.141516 deve ser exibido 3.
6. Uma prática adequada em normalizações consiste em garantir que o denominador não seja zero. Entretanto, nos exercícios desta lista isso não é necessário, pois as entradas já garantem essa condição.

Redução de Dimensionalidade

1. (Atividade Variância Explicada) Construa um programa que receba três inteiros s ($0 \leq s \leq 1024$), n ($0 \leq n \leq 10^3$) e m ($0 \leq m \leq n$), indicando uma **seed** para geração pseudoaleatória de números, número de amostras e a dimensão dos dados, nesta ordem. Em seguida, leia um número real $v \in [0, 1]$. A partir desses dados, o programa deve calcular o PCA utilizando SVD (conforme visto em aula). Em seguida, o programa deve exibir a menor quantidade de componentes principais necessária para que a variância explicada acumulada seja maior ou igual a v . Por exemplo, considerando $m = 5$ e $v = 0.8$, se as proporções de variância explicada forem 0.5, 0.2, 0.15, 0.1e0.05, então o programa deve exibir 3, pois a soma dos três primeiros componentes é $0.85 \geq v$.

Observações. Diferentemente dos outros problemas, aqui os dados são gerados de forma pseudoaleatória a partir da **seed** fornecida, conforme ilustra parte do código abaixo; desta forma, é importante que você não altere esse trecho. Para o cálculo do SVD, utilize `np.linalg.svd(., full_matrices=False)`. Para o cálculo da variância utilize apenas Σ^2 . Não altere o processo de centralização dos dados.

```
1 import numpy as np
2 import random
3
4 np.random.seed(s)
5 random.seed(s)
6
7 X = np.random.randn(n, m)
8 Xc = X - X.mean(axis=0)
```

As tabelas abaixo apresentam exemplos de entradas e as respectivas saídas do programa.. Note que, para obter as saídas esperadas, é preciso executar o geração pseudoaleatória conforme o código acima.

Entrada	Saída
0 100 10	10
0.99	

Entrada	Saída
64 200 12	11
0.95	

2. (Atividade Violação da Ordem de Distâncias) Construa um programa que receba quatro inteiros s ($0 \leq s \leq 1024$), n ($0 \leq n \leq 10^3$), m ($0 \leq m \leq n$) e c ($0 \leq c < m$), indicando uma **seed** para geração pseudoaleatória de números, o número de amostras, a dimensão dos dados e o número de componentes do PCA, nesta ordem. A partir dessas entradas, o programa deve calcular o PCA utilizando SVD (conforme visto em aula) e exibir a quantidade de triplas (i, j, k) para as quais a seguinte condição **não** é satisfeita: $\mathbf{1}(\|x_i - x_j\|_2 < \|x_i - x_k\|_2) = \mathbf{1}(\|\hat{x}_i - \hat{x}_j\|_2 < \|\hat{x}_i - \hat{x}_k\|_2)$, para todo i, j, k distintos. Nessa formulação, x e $\hat{x}$ correspondem, respectivamente, aos dados no espaço original de dimensão m e aos dados projetados nos k componentes principais do PCA.

Observações. Diferentemente dos outros problemas, aqui os dados são gerados de forma pseudoaleatória a partir da **seed** fornecida, conforme ilustra parte do código abaixo; desta forma, é importante que você não altere esse trecho. Para o cálculo do SVD, utilize `np.linalg.svd(., full_matrices=False)`. Para o cálculo da distância entre duas amostras, use `np.linalg.norm(., ord=2)`. Não altere o processo de centralização dos dados nem a ideia geral da contagem já implementada.

```
1 import numpy as np
2 import random
3 from itertools import permutations
4
5 np.random.seed(s)
6 random.seed(s)
7
8 X = np.random.randn(n, m)
9 Xc = X - X.mean(axis=0)
10
11 cont = 0
12 for i, j, k in permutations(range(n), 3):
13     xi, xj, xk = X[i], X[j], X[k]
14
15     if ??? != ???:
16         cont += 1
```

As tabelas abaixo apresentam exemplos de entradas e as respectivas saídas do programa.. Note que, para obter as saídas esperadas, é preciso executar a geração pseudoaleatória conforme o código acima.

Entrada	Saída
0 15 5 2	528

Entrada	Saída
64 20 12 2	1506